OSNUTEK

Izpit – VS – 16. 6. 2016 – (1)

1. **Kaj je datoteka?**

Datoteka je osnovni objekt datotečnega sistema. Je podatkovna struktura sekvenčnega značaja (podatki si sledijo drug za drugim). Osnovni operaciji sta *read* in *write*, pred pisanjem in branjem je potrebno datoteko odpreti ali ustvariti. Položaj vzdolž datoteke beleži datotečni kazalec.

   a. **Navedite tipe datotek v sistemu Linux in poudarite njihove glavne lastnosti.**

- Navadna datoteka: ASCII ali binarna
- Direktoriji: na disku so shranjeni kot navadne datoteke, a imajo dve pomembni razliki:
  - Vsebina je dosegljiva preko v naprej določenih operacij za delo z direktoriji
  - Vsaka sprememba se takoj zabeleži na disk
- Bločne in znakovne naprave: za komunikacijo s strojno opremo, nahajajo se v mapi /dev
- FIFO ali poimenovana cev: povezava med dvema nepovezanima procesoma na istem računalniku. Podatki so lahko prebrani le enkrat v istem vrstnem redu kot so bili zapisani
- UNIX vtičnice: za povezave med oddaljenimi procesi
- Simbolična povezava: datoteka, ki kaže na datoteko s podatki ali direktoriji

2. **Datoteka v sistemu Linux vsebuje 16 podatkov tipa *uint32_t*. Vrednosti podatkov so zapisane binarno in so naslednje: 1, 2, 3, 4, ..., 16. Isto datoteko odpreta dva procesa. Dostop do datoteke poteka takole:**
   a. Najprej prvi proces odpre datoteko z:
      fd = open(imeDatoteke, ORDONLY);
   b. Takoj zatem odpre isto datoteko drugi proces:
      fd = open(imeDatoteke, ORDONLY);
   c. Nato prvi proces prebere podatke z:
      read(fd, Buf, sizeof(uint32_t));
      Definicija spremenljivke (polja) *Buf* je *uint32_t Buf [16]*.
   d. Takoj zatem drugi proces prebere podatke z
      *read(fd, Buf, 2 * sizeof(uint16_t));*
      Definicija spremenljivke (polja) *Buf* je *uint16_t Buf[16]*.

   Odgovorite na naslednja vprašanja:
   a. Kolikšnaa je dolžina datoteke, izražena v bajtih? 16*4 byte = 64 bytov
   b. Koliko prostora (na disku) zavzame vsebina te datoteke in zakaj? 4K
   c. Kakšne so vrednosti komponent polja *Buf* prvega procesa?
   d. Kakšne so vrednosti komponent polja *Buf* drugega procesa? Mogoča sta dva odgovora. Napišite oba in ju komentirajte.

3. **Isto datoteko odpreta dva procesa. Dostop do datoteke poteka takole:**
   a. Najprej prvi proces odpre datoteko z:
      fd = creat(imeDatoteke, 0644);
   b. Kmalu zatem odpre isto datoteko drugi proces:
      fd = open(imeDatoteke, ORDONLY);
   c. Nato prvi proces zapiše podatke z

write(fd, Buf, sizeof(Buf));
Definicija spremenljivke (polja) *Buf* je *uint32_t Buf [4] = {1, 2, 3, 4}.*

d. Potem, ko prvi proces zapiše podatke, jih drugi proces prebere z
read(fd, Buf, sizeof(Buf));
Definicija spremenljivke (polja) *Buf* je *uint16_t Buf[4]* .

Odgovorite na naslednja vprašanja:

a. Kakšna je dolžina datoteke *imeDatoteke,* izražena v bajtih?
b. Koliko prostora (na disku) zasede vsebina te datoteke ter zakaj?
c. Kakšne so vrednosti komponent polja *Buf* prvega procesa?
d. Kakšne so vrednosti komponent polja *Buf* drugega procesa? Mogoča sta dva odgovora. Napišite oba in ju komentirajte.

4. Kaj so cevi (pipes)?

Je enosmerni komunikacijski kanal, ki lahko obstaja med dvema ali več procesi v sorodstvu. Za obojestransko komunikacijo sta potrebni dve cevi.

a. Kakšne tipe cevi poznate in kakšne so razlike med njimi?

Poimenovane cevi FIFO in navadne cevi. Razlika med njimi je:
Poimenovane: za povezavo med procesoma, ki nista v sorodstvu
Navadne cevi: za povezavo med dvema ali več procesoma v sorodstvu

b. Kako načeloma napeljemo cev med procesoma v sorodstvu?

Cev ustvarimo v roditelju z ukazom *pipe()* in nato ustvarimo nov proces z ukazom *fork()*.

c. Kaj se zgodi v primeru, ko je cev 'polna'? Kdaj do tega pride?

Ustavi pisanje v cev, dokler proces, ki bere ne začne brati podatke iz nje.

d. Kaj se zgodi v primeru, ko je cev 'prazna'? Kdaj do tega pride?

Proces, ki bere čaka na proces, ki piše.

e. Ali lahko iz cevi, v katero en proces pošilja (piše) hkrati sprejemata (bereta) dva procesa. Zakaj?

Da. Aplikaciji je prepuščeno, kako naj se vrstijo operacije branja.

f. Ali lahko v cev, iz katere bere en proces, hkrati pišeta dva procesa? Zakaj?

Da. Aplikaciji je prepuščeno, kako naj se vrstijo operacije pisanja. Cev poskrbi, da se zaporedni vpisi različnih procesov zgodijo zaporedno. Tako, se posamični vpisi različnih procesov med sabo zagotovo ne bodo prepletali, če le niso preveliki.

g. Kako proces, ki sprejema (bere), ugotovi, da je cev na drugi strani zaprta?

Proces z ukazom *read()* prebere 0.

h. Kaj se zgodi, ko proces piše v cev, ki je na drugi strani zaprta?

Nadaljnje, pisanje v cev z zaprtim deskriptorjem za branje ima za posledico pošiljanje signala, ki sporoča, da je bila cev »pretrgana«.

i. Kako načeloma napeljemo cev med procesoma, ki nista v sorodstvu?

Cev napeljemo tako, da najprej ustvarimo FIFO (poimenovano cev) z ukazom mkfifo -m, dodelimo pravice za pisanje in branje in dodelimo ime. Nato pa v obeh procesih kličemo ustvarjeni FIFO z ukazom open.

5. Kaj je računalniški proces?

Program v izvrševanju.

a. Kako nastane nov proces?

Tako, da ga ustvari že obstoječi proces.

b. Skicirajte diagram prehajanja stanj procesa in ga na kratko

obrazložite.

c. **V kakšnih načinih se izvaja proces in kakšna je razlika med načini? Kdaj pride do spremembe načina?**

V uporabniškem ali sistemskem načinu. Razlika je, da večino procesorskega časa smo v uporabniškem načinu. Ko se izvede sistemski klic, pa preide izvajanje procesa v sistemski način, kjer se izvede izvajanje željene sistemske operacije. Nato preide proces nazaj v uporabniški način.

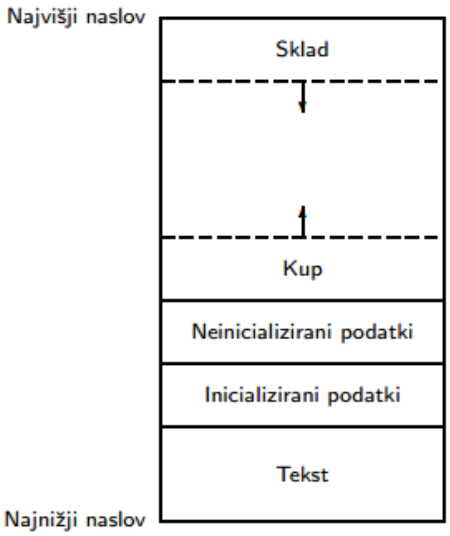
d. **Kaj je kontekstni oziroma pomenski preklop in kdaj do njega pride? Ka- tera del programske opreme opravi pomenski preklop?**

Kontekstni oz. pomenski preklop je preklapljanje iz uporabniškega v sistemski način izvajanja procesa in obratno. Pride do tega takrat, ko se v procesu izvede sistemski klic.

e. **Kako nastane (nov) proces in kako se inicializira (naloži v pomnilnik)?**

Ustvari ga že obstoječi proces. Proces se inicializira z ukazom *exec()*.

f. **Skicirajte načelno zgradbo procesa v pomnilniku in jo komentirajte.**



Delu procesa z ukazi, torej ukaznemu segmentu pravimo tekst. Sledi segment inicializiranih podatkov (podatki, ki imajo deifinirane vrednosti že z izvorno kodo). Inicializirajo se ob nalaganju programa v pomnilnik. Dinamično alocirani podatki se nahaja v skladu. Dosegljivi so samo znotraj dotične funkcije in veljavni tedaj, kadar se funkcija izvršuje. Neinicializirani podati so podatki, ki jim je pomnilnik sicer dodeljen statično, a sprva pri nalaganju programa v pomnilnik nimajo še definiranih vrednosti. V kopici (Kup) se pomnilnik zaseže dinamično, tako kot narekujejo potrebe med izvrševanjem programa.

g. **Kaj je logični (virtualni) naslovni prostor in kaj je fizični naslovni prostor?**

Fizični naslov je naslov pomnilniške besede glavnega pomnilnika, to je pomnilniške besede tistega pomnilnika, do katerega ima procesna enota naključni dostop. Pomnilniška beseda ima naslov in vsebino.
Logični naslovi pripadajo programu oziroma procesu. Je naslov znotraj programa, to je naslov ukaza ali podatka. Množica logičnih naslovov procesa sestavljajo logično naslovno področje.

h. **Kje v logičnem naslovnem prostoru bi bila spremenljivka *static int x = 100;*?**

V inicializiranih podatkih.

i. **Kje bi bila spremenljivka *static int y;*?**

V neinicializiranih podatkih.

j. **Kje bi bila spremenljivka *int z;*, če je definirana znotraj funkcije *main()*?**

V skladu.

k. **Kje bi bil dodeljen pomnilnik za spremenljivko *niz = malloc(512);*?**

V kopici.

l. **Kje bi bil dodeljen pomnilnik, ki si ga deli več procesov? Ali je pomnilnik z deljenim dostopom v okviru jedra?**

Deljen pomnilnik se nahaja med naslovnim prostorom kopice in sklada. Da, zato ker sistemsko jedro dodeli isti del fizičnega pomnilnika.

6. **Kaj je nit in kaj je večnitnost?**

Je zaporedje ukazov, ki jih procesna enota izvršuje drugega za drugim. Večnitnost je zmožnost programa, da navidezno hkrati izvaja več niti enega procesa.

a. **Katere so glavne razlike med večopravilnostjo in večnitnostjo?**

Večopravilnost je zmožnost operacijskega sistema za navidezno izvajanje več procesov hkrati, med tem ko pa je večnitnost je zmožnost programa, da navidezno hkrati izvaja več niti enega procesa

b. **Navedite glavne pripomočke za sinhronizacijo procesov in niti.**

- Niti: pogojne spremenljivke, *pthread_mutex* (funkcija za preprečevnje sočasnega dostopa do sredstev, ki ne dovoljujejo sočasnega dostopa), *pthread_rwlock* (funkcija za preprečevanje sočasnega pisanja in branja skupne spremenljivke)
- Procesi: včasih tudi signali, semafor

7. **Kaj je semafor?**

Pripomoček za sinhronizacijo procesa.

a. **Z uporabo semaforjev zagotovite, da se bo dostop procesov *Pi, (i = 0, 1, 2,..., N)* do datoteke *D* vrstil krožno, t.j. *0,1,2,...,N, 0,1,2,0,1,2,....,N*.**

8. **Opišite koncept komunikacije dveh procesov s pomočjo deljenega pomnilnika.**

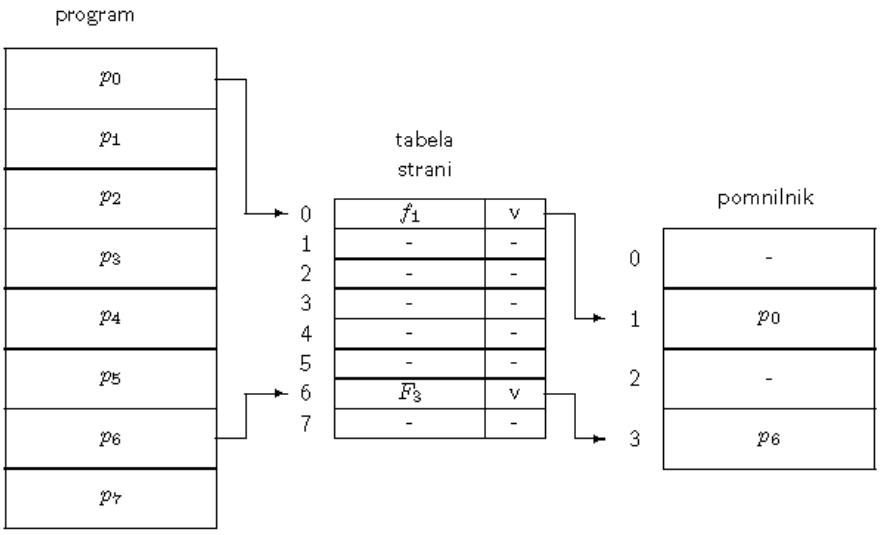
V sistemu z deljenim pomnilnikom poteka komunikacija prek pomnilnika oziroma dela pomnilnika, do katerega iam dostop večje število procesov. Namesto deljeni pomnilnik lahko rečemo tudi skupni pomnilnik. Sistemsko jedro namreč več procesom dodeli isti del fizičnega pomnilnika. Ko je pomnilnik dodeljen, poteka komunikacija med procesi direktno brez posredovnja jedra. Predstavlja najhitrejšo obliko medprocesnega komuniciranja, saj do prenašanja podatkov v pravem pomenu besede sploh ne pride.

a. **Kako poteka komunikacija (prek pomnilnika) med nitmi istega procesa?**

Niti si delijo isti naslovni prostor procesa, kar pomeni, da si delijo pomnilnik dodeljen procesu.

b. **Nakažite koncept izvedbe deljenega pomnilnika v sistemu z ostranjenim**

navideznim pomnilnikom.



Slika 4.3: *Koncept ostranjenega navideznega pomnilnika. Strani programa se preko tabele strani preslikajo v okvirje strani. V konkretnem primeru sta v pomnilnik nameščeni samo dve od osmih strani programa. Stran 0 je nameščena v okvir 1 in stran 6 je nameščena v okvir 3.*

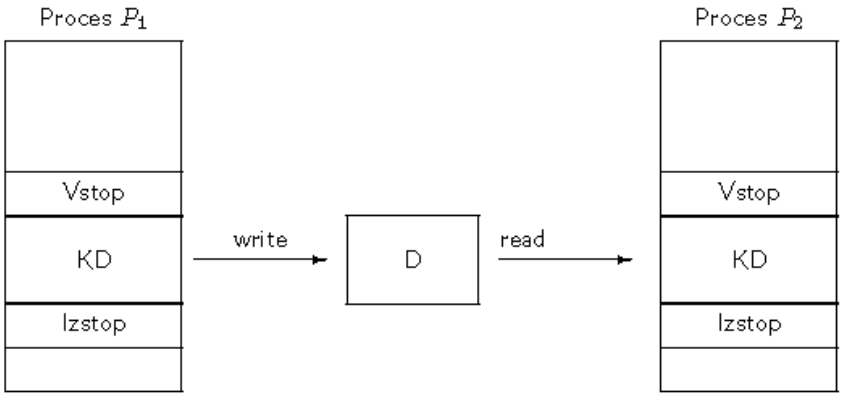
c. Razložite razliko med logičnimi in fizičnimi naslovi?

Logični naslovi pripadajo programu oziroma procesu, ki se preslikajo v fizične naslove pomnilnika.

9. Kaj je kritičo področje?

Ko se en proces izvaja v tistem delu, ki posega po datoteki, se drug proces ne sme izvajati v tistem njegovem delu, ki posega po tej isti datoteki, sicer pa se lahko izvaja brez omejitve. Delu programa oziroma procesa, ki dostopa do skupnega sredstva, pravimo kritični del ali kritično področje.

a. Nakažite problem pri dostopu procesov (ali niti) do skupnega sredstva.



Slika 7.3: *Problem dostopa dveh procesov do skupnega sredstva (datoteke). Tedaj, ko proces P_1 dostopa do datoteke in se torej izvaja v kritičnem delu (KD), moramo preprečiti dostop do datoteke procesu P_2 ali povedano drugače, preprečiti mu moramo vstop v kritični del dokler preces P_1 ne zapusti svojega kritičnega dela.*

b. Kaj razumete pod pojmom atomičnost operacije?

Ko se operacija enkrat začne, se mora izvršiti do konca brez prekinitve.

c. Nakažite rešitev problema.

Splošna rešitev problema kritičnega področja mora zadostiti zahtevi po medsebojnem izključevanju in še nekaterim drugim zahtevam:

- Medsebojno izključevanje v vseh okoliščinah
- Ne sme priti do zastoja
- Ne sme priti do nepredvidljivo dolgega odlaganja
- Izvajanje enega procesa znotraj kritičnega dela ne sme ovirati napredovanja drugih procesov izven kritičnega področja

10. Razložite signale v sistemu Linux

Signal je naznanilo procesu, da se je nekaj zgodilo. Signal lahko pošlje sistemsko jedro, proces drugemu procesu ali proces samemu sebi. Proces, ki signal prejme, lahko signal obravnava na privzet način, ki pomeni takojšen zaključek procesa. Izjemoma sme proces signal prezreti.

a. Čemu služijo funkcije *signal, kill, pause* in *alarm*

Signal – funkcija določi, kako naj proces signal obravnava oziroma kako naj ga prstreže

Kill – funkcija pošlje signal procesu ali skupini procesov. Številko procesa podamo z argumentom pid.

- Pid>0: pošlje signal temu procesu
- Pid=0: signal se pošlje vsem procesom iste skupine
- Pid=-1: signal se pošlje vsem procesom, katerim sme proces poslati
- Pid<-1: signal se pošlje vsem procesom

Pause – s funkcijo *pause* je procesu moč čakati na sprejem signala, torej vezati svoje nadaljevanje na nek dogodek v drugem procesu

Alarm – s to funkcijo je moč upravljati časovnik procesa. Klic nastavi začetno vrednost časovnika. Ko se časovnik izteče, dobi proces signal SIGALRM in navadno konča

b. Kako se lahko konča proces in kako je moč ugotoviti kako se je končal?

Proces se lahko konča s funkcijo *abort*. Še pred zaključkom proces generira datoteko z imenom *core*, v katero shrani posnetek stanja procesa v trenutku sprejema signala.

11. Pojasnite sistem komunikacijskih vtičnic.

Sistem vtičnic omogoča enotno obravnavanje medprocesne komunikacije neodvisno od krajevne porazdelitve procesov zatorej predstavlja bistveno razširitev zmožnosti medprocesnega komuniciranja. Služijo dvosmernemu komuniciranju med procesi, ki so poljubno porazdeljeni v omrežju.

a. Pojasnite načeli (pravili) debelega in tankega konca. Katero pravilo je v veljavi v omrežju?

Gre za različne interpretacije vrstnega reda bajtov. Debel konec pomeni zaključek na višjem naslovu, tanek konec pa zaključek na nižjem naslovu. Omrežje uporablja pravilo debelega konca.

b. Kaj sestavlja naslov vtičnice v komunikacijski domeni AF INET? Kako je to povezano s protokoli.

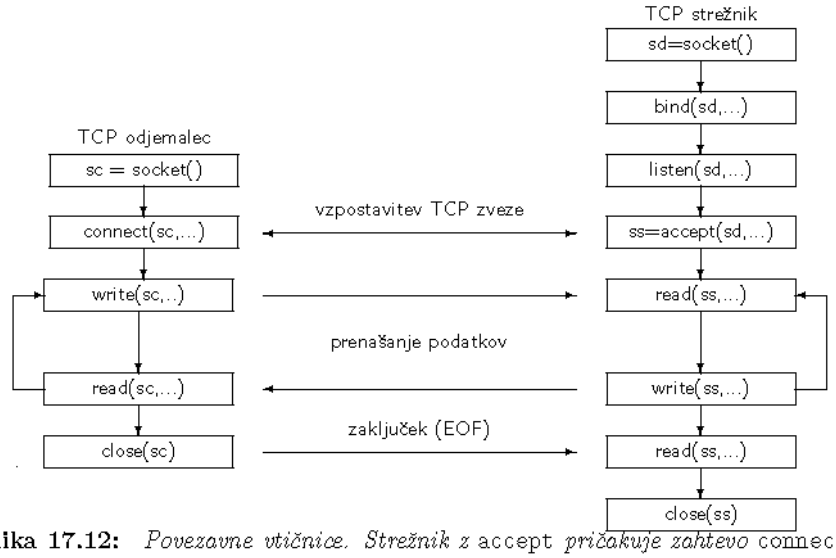
Naslov vtičnice sestavljata številka vrat (TCP ali UDP), na katera je pripet proces ter omrežni naslov (IPv4) vozlišča. Pri ukazu *socket* podamo prvi argument AF_INET za komunikacijski protokol. Naslednji argument pa je tip vtičnice, ki je vezan na komunikacijski protokol in so lahko naslednji SOCK_STREAM (TCP), SOCK_DGRAM (UDP) in SOCK_SEQPACKET (SCTP)

c. Kakšna je razlika med povezavnim in nepovezavnim tipom vtičnic?

Pri povezavnem tipu vtičnic (TCP), je povezava vzpostavljena med strežnikom in odjemalcem, ter podatki se prenašajo po ustaljeni poti. Pri vsakem prejetem paketku odjemalec odgovori, da je paketek prejel in šele nato strežnik pošlje naslednji paketek.

Pri nepovezavnem tipu vtičnic (UDP) do povezave ne pride. Paketki ne prehajajo po isti poti, strežnik ne čaka na odgovor odjemalca temveč neprestano pošilja paketke in podatki ni nujno, da pridejo po istem vrstnem redu, kot so bili poslani.. Prenos podatkov je hitrejši kot pri povezavnem načinu.

d. Skicirajte zaporedje korakov v primeru komunikacije med povezavnima strežnikom in odjemalcem.



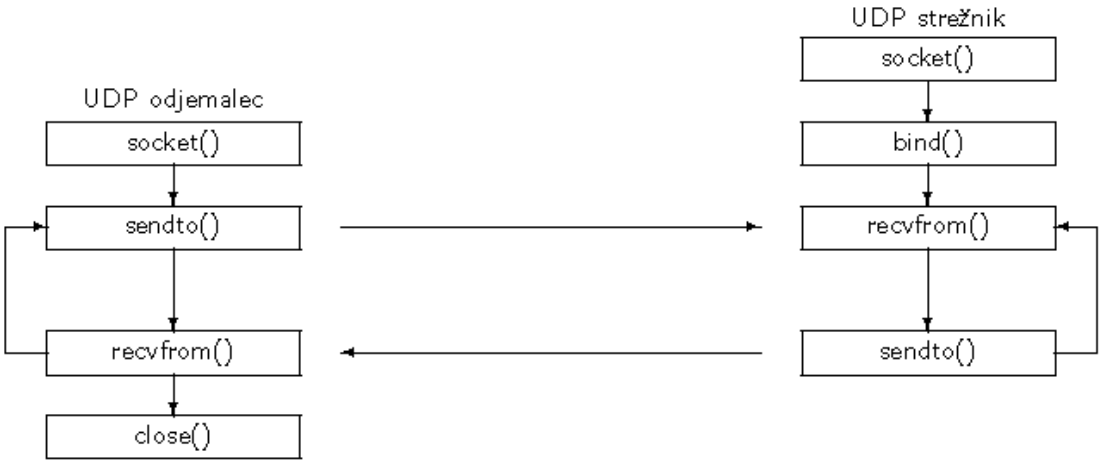
Slika 17.12: *Povezavne vtičnice. Strežnik z* `accept` *pričakuje zahtevo* `connect` *odjemalca za vzpostavitev zveze. Ko je zveza vzpostavljena, teče prenos podatkov v obe smeri, dokler se povezava ne zapre oziroma sprosti.*

e. Po katerem protokolu poteka prenos?

TCP

f. Komentirajte komunikacijo odjemalec / strežnik z vidika uporabljenega protokola.

g. Skicirajte zaporedje korakov v primeru komunikacije med nepovezavnima strežnikom in odjemalcem.



Slika 17.13: *Nepovezavne vtičnice prek protokola UDP. Strežnik z* `recvfrom` *pričakuje podatke od potencialnega odjemalca in se po potrebi s* `sendto` *odzove.*

h. Po katerem protokolu poteka prenos?

UDP

i. Komentirajte komunikacijo odjemalec / strežnik z vidika uporabljenega protokola.

12. Kako 'merimo' čas v računalniku in kakšne vrste časa poznamo?

Realni čas in procesorski čas. Realni čas je tisti čas, v katerem živimo in ga merimo od izbranega trenutka naprej. Procesorski čas je vezan na napredovanje procesa. Je tisti čas, ko je procesor dodeljen procesu. Vsak proces ima vsaj dva časovnika, eden šteje od nastanka do konca procesa, drugi pa se povečuje le tedaj, ko proces teče. Naprej delimo še na uporabniški in sistemski čas.

Vsak računalniški sistem ima vsaj en časovni števec oziroma elektronsko vezje, ki služi za referenco pri merjenju časa, tako realnega kot procesorskega. Števec se enakomerno povečuje z urnim taktom.

a. Navedite glavne funkcije časovnikov s primeri uporabe.

Time – vrne število sekund od prelomnega datuma do klica.

gettimeofday – vrne koledarski čas v strukturi na katero kaže.

times – dostop do procesorskega časa. Vrne število udarcev ure, ki je poteklo od referenčnega trenutka enkrat v preteklosti (npr. od zagona sistema)

clock – funkcija vrne celoten procesorski čas procesa

13. Kaj so sistemi realnega časa?

Sistemi ki delujejo v realnem času. Sistemi realnega časa so računalniški sistemi z določenimi lastnostmi, ki jih vsi računalniški sistemi nimajo. Spoznamo jih po tem, da je njegovo delovanje neposredno povezano z realnim časom. Takšni sistemi so naprimer multimedijski sistemi.

a. Kakšne vrste sistemov realnega časa poznamo?

Sisteme, v katerih smejo biti nekatere časovne zahteve občasno kršene imenujemo sistemi realnega časa z mehkimi oz. ohlapnimi omejitvami. (avtomobilska okna, multimedijske naprave)

Časovno kritični sistemi, v katerih nobena časovna zahteva ne sme biti nikoli kršena, imenujemo sistemi realnega časa s trdimi oz. strogimi omejitvami. (airbag, sistemi za vodenje in nadzor kemijskih ali fizikalnih procesov, vojaških sistemov)

b. Navedite po en primer za vsako vrsto sistemov realnega časa in utemeljite zakaj so to sistemi realnega časa izbrane kategorije.

c. Naštejte bistvene lastnosti operacijskih sistemov za realni čas.

- Večopravilnost, prioritetno razvrščanje opravil
- Večnitnost in prioritetno razrvrščanje niti
- Prednostno razvrščanje ali predopravilnost
- Podpora medprocesorskim komunikacijam, deljenemu pomnilniku in sinhronizaciji opravil
- Asinhrono odzivanje na zunanje dogodke z vnaprej znanimi zakasnitvami
- Zaklepanje procesov v pomnilniku

d. Navedite nekaj primerov operacijskih sistemov za realni čas.

windowsC, VxWorks, freertos, eCos, uOS,

e. Opišite razvrščanje opravil realnega časa v sistemu Linux.

Razvrščevalnik sestavlja časovni razpored opravil in dodeljuje procesno enoto. Ko razvrščevalnik odvzame procesor enemu procesu ter ga dodeli drugem procesu pravimo, da opravi menjavo procesa oziroma pomenski preklop.

14. Naštejte in na kratko opišite tipična pravila razvrščanja, ki se uporabljajo v operacijskih sistemih.

- **Krožno razvrščanje opravil:** v sistemu s krožnim razvrščanjem dobi vsak proces na razpolago enak in v naprej predpisan časovni interval (kvant oz. rezino). V primeru, da

proces potrebuje manj časa, se procesor takoj dodeli naslednjemu procesu. Če pa proces, ki se izvaja, po poteku določenega časa še ni končan, se mu procesna enota odvzame, možnost za napredovanje dobi naslednji proces in tako dalje v krožnem zaporedju.

- **krajše opravilo prej:** pri takem razvrščanju krajše opravilo prej dobi procesno enoto. Najprej najkrajši proces, nato naslednji najkrajši in nazadnje najdaljši proces. Dejansko gre za obliko prioritetnega razvrščanja, pri čemer je prioriteta procesa obratno sorazmerna njegovemu trajanju.
- **Prioritetno razvrščanje:** v sistemih s prioritetnim razvrščanjem imamo končno število prioritetnih nivojev (32 ali več) in vsakemu procesu oz. opravilu je prirejena prioriteta. Procesi si delijo procesno enoto na osnovi dodeljenih prioritet. V sistemu s predopravilnostjo je zagotovljeno, da se bo v vsakem trenutku izvajal proces z najvišjo prioriteto
- **Pogostejše opravilo prej:** pomembno je za sisteme v realnem času. V sistemih realnega časa je velika večina opravil periodičnih ali pa se da njihovo pogostost vsaj oceniti
- **Najkrajši skrajni rok:** spada med algoritme razvrščanja na osnovi dinamičnih prioritet. Po tem pravilu se prioritete opravil spreminjajo s časom v odvisnosti od skrajnih rokov opravil. Razvrščevalnik v danem trenutku dodeli procesor tistemu opravilu, ki ima najkrajši skrajni rok

15. Razložite obrat prioritet in navedite rešitev problema.

Ko proces z nižjo prioriteto zasede sredstvo, ki ne dovoljuje sočasnega dostopa. Dostop do sredstva bi lahko uredili na primer s semaforjem ali z mutexom. Obratu prioritet se izognemo z dedovanjem prioritet, to pomeni, da vsi procesi, ki dostopajo do skupnega sredstva, v času dostopa začasno dobijo prioriteto procesa, ki ima izmed njih najvišjo prioriteto.
Druga rešitev je dvih prioritete procesu, ki zaseže skupno sredstvo na najvišjo prioriteto.

16. Opišite načelno delovanje razvrčevalnika v sistemu Linux.

a. Opišite načelno pravilo razvrščanja navadnih opravil v sistemu Linux.

17. Kaj se dogaja? Kaj se izpiše?

```
int main( int n, char **argv )
{
        int s;
        printf("%s \n", argv[0]);
        execlp(argv[0], "ls", (char *)NULL);
        wait( &s );
        exit( 1 );
}
```

18. Kaj se dogaja? Kaj se izpiše?

```
int main( int n, char **argv )
{
        int s;
        printf("%s \n", argv[0]);
        execlp("ls", argv[0], (char *)NULL); wait( &s );
        exit( 1 );
}
```

19. Kaj se dogaja? Kaj se izpiše?

```
int main( int n, char **argv )
{
        int s;
        printf("%s \n", argv[0]);
        execlp(argv[0], argv[0], (char *)NULL);
        wait( &s );
        exit( 1 );
}
```

20. Komentirajte spodnji program in razložite kaj se dogaja ob izvršitvi.

```
int main( void )
{
        pidt pid;
        int s;
        while(1)
        {
                pid = fork( );
                wait( &s );
                exit( 1 );
        }
}
```

21. Komentirajte spodnji program in razložite kaj se dogaja ob izvršitvi.

```
int main( void )
{
        pidt pid;
        int s;
        while( fork() > 0 )
        {
                wait( &s );
        }
        exit( 1 );
}
```

22. Kaj se dogaja?

```
int main( int n, char **argv)
{
        int s;
        execlp(argv[0], argv[0], (char *)NULL);
        wait( &s );
        exit( 1 );
}
```

23. Kaj se dogaja? Koliko procesov imamo? Domnevamo, da ne pride do napake. Kaj se izpiše?

```
int main( void )
{
        if( fork() == -1) exit(1);
        if( fork() == -1) exit(2);
        if( fork() == -1) exit(3);
        printf("Pid: %d \n", getpid() );
        exit( 0 );
}
```

24. Kaj se dogaja? Koliko procesov imamo? Domnevamo, da ne pride do napake. Kaj se izpiše?

```
int main( void )
{
        pidt pid;
        if( (pid = fork( )) == -1) exit(1);
        if( (pid = fork( )) == -1) exit(2);
        if( (pid = fork( )) == -1) exit(3);
        printf("Pid: %d, %d \n", getpid(), pid );
        exit( 0 );
}
```

25. Kaj se dogaja?

```
int main( int n, char **argv)
{
        pidt pid;
        int s;
        if( (pid = fork()) > 0)
                execlp(argv[0], argv[0], (char *)NULL);
        wait( &s );
        exit( 1 );
}
```

26. Kaj se dogaja?

```
int main( int n, char **argn )
{
        pid_t pid;
        int s;
        if( (pid = fork()) == 0)
                execlp(argn[0], argn[0], (char *)NULL);
        wait( &s );
        exit( 1 );
}
```